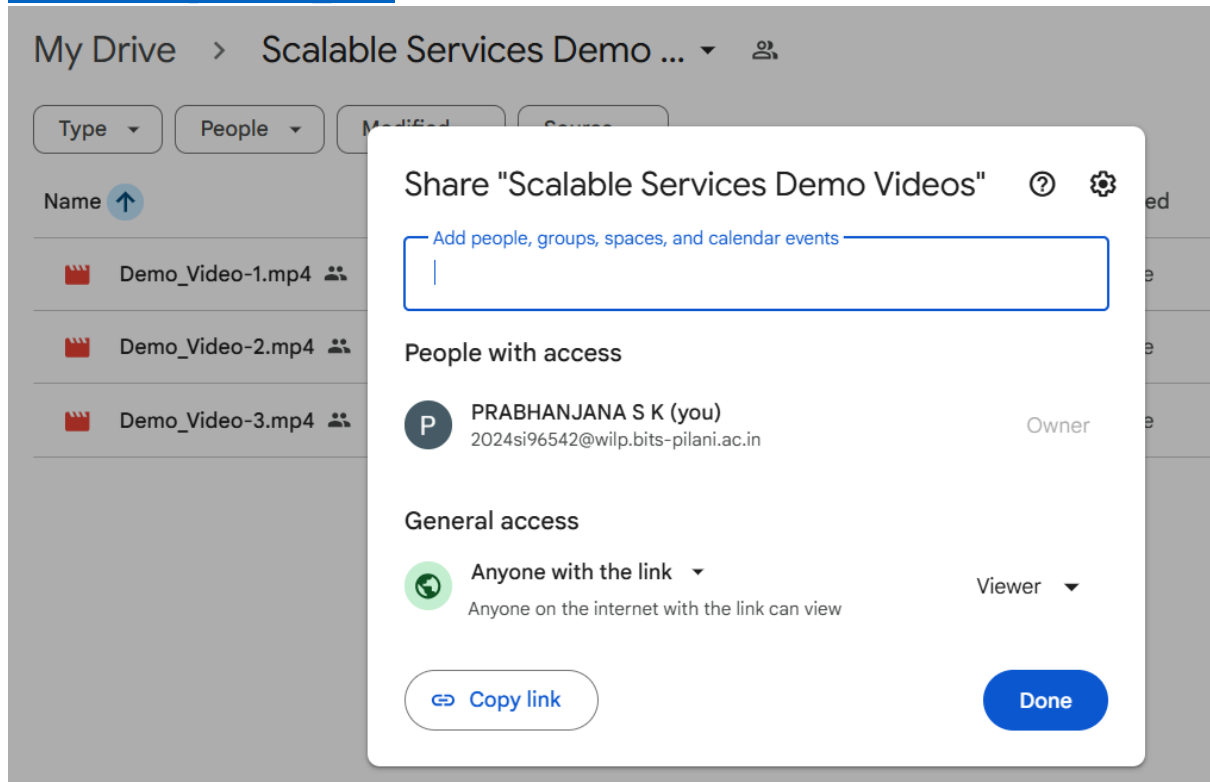


Scalable Services- Project Documentation

Application Name: University Enrolment System

Demo Video link:

[https://drive.google.com/drive/folders/1f1pImIZ3EQiEPZ8gAJLodvpmUYkV45wI?usp=drive link](https://drive.google.com/drive/folders/1f1pImIZ3EQiEPZ8gAJLodvpmUYkV45wI?usp=drive_link)



1. Group Details & Contribution

Prabhanjana SK - 2024SI96542

Contributions: Architecture Design, API Gateway, Docker/Containerization, Student, Course & Enrolment Services, Integration Testing, version control.

Shankar A - 2024SI96511

Contributions: Database Implementation, Error Handling

2. Brief Application Description

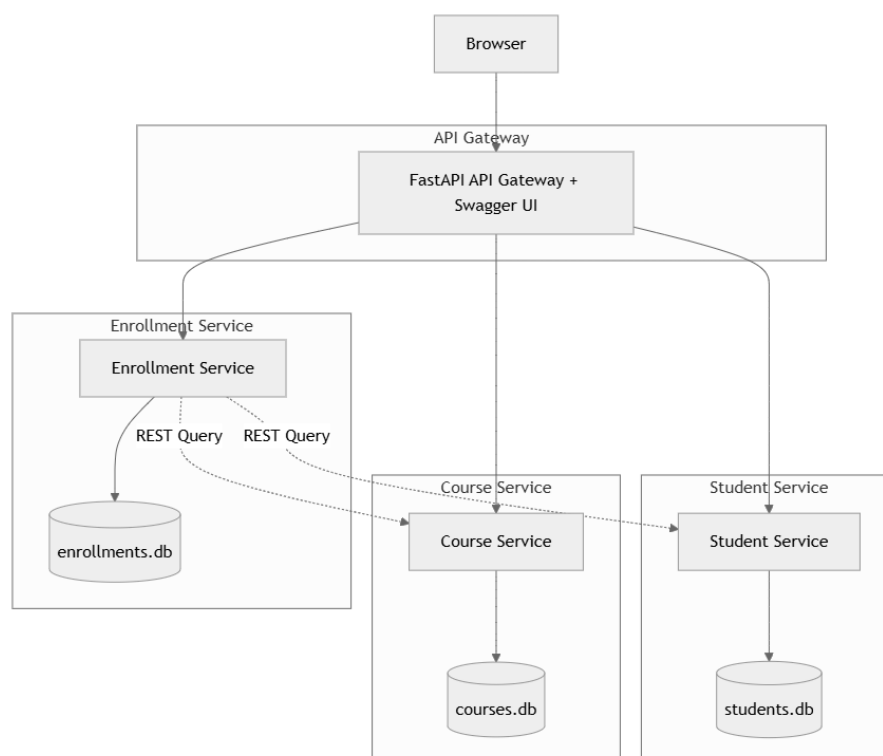
The application is a distributed **University Enrollment System** designed to manage educational records. It allows administrators to securely create and manage student profiles, maintain a catalog of offered courses, and process student enrollments into those courses while enforcing strict data validation.

3. Architecture of the Application

The system transitions away from a monolithic structure into a fully containerized Microservices Architecture. It consists of four primary components:

1. **API Gateway:** The centralized entry point for all client requests.
2. **Student Service:** Manages student identity and profiles.
3. **Course Service:** Manages course offerings and details.
4. **Enrolment Service:** Manages the relationships (registrations) between students and courses.

All services are deployed within isolated Docker containers and communicate over a private, internal Docker bridge network.



4. Service Decomposition Rationale

The application was decomposed based on **Business Domain-Driven Design**.

By isolating students, courses, and enrollments into their own autonomous services, we achieved:

- **Independent Scalability:** If registration week causes a spike in enrollment traffic, the Enrollment Service can be scaled horizontally (as demonstrated in Demo Video 3) without wasting resources scaling the Student or Course services.
- **Fault Isolation:** A crash in one domain (e.g., the Course database going offline) will not bring down the entire system. The API Gateway and other services remain alive and gracefully handle the failure.

5. Service API Design

The system exposes RESTful APIs. Below is a summary of the core endpoints routed through the API Gateway (<http://localhost:8080>):

Student Service (/api/v1/students)

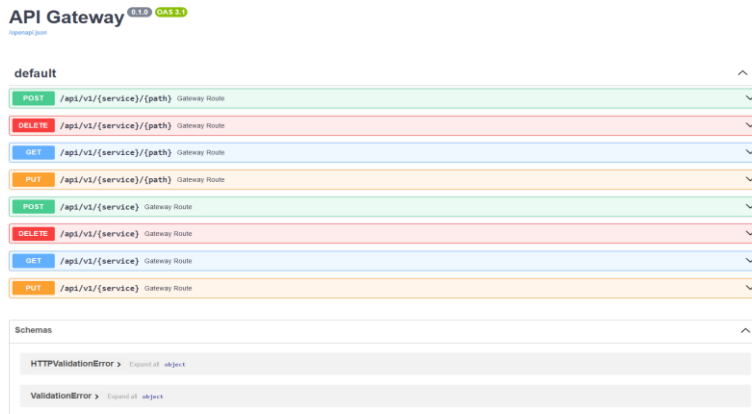
- GET / - Retrieve all students
- GET /{id} - Retrieve specific student
- POST / - Create a new student
- PUT /{id} - Update student details
- DELETE /{id} - Remove a student

Course Service (/api/v1/courses)

- GET / - Retrieve all courses
- GET /{id} - Retrieve specific course
- POST / - Create a new course
- PUT /{id} - Update course details
- DELETE /{id} - Remove a course

Enrollment Service (/api/v1/enrollments)

- GET / - Retrieve all enrollments
- POST / - Create a new enrollment (Validates Student/Course IDs)
- PUT /{id} - Update an enrollment
- DELETE /{id} - Remove an enrolment



6. Database Design and Pattern Used

The architecture strictly enforces the **Database-per-Service** pattern to guarantee data isolation and loose coupling.

- **Technology:** SQLite (Local file-based databases).
- **Implementation:** The Student Service owns students.db, Course owns courses.db, and Enrollment owns enrollments.db.
- **Pattern Rules:** No service is allowed to directly query or modify another service's database. All data sharing must occur through network API calls.

7. Communication Style Used Between Services

The system utilizes **Synchronous HTTP/REST Communication**.

When an enrollment is created, the Enrollment Service acts as a client. It uses asynchronous HTTP requests (httpx) to send synchronous GET requests to the Student Service and Course Service. It waits for a 200 OK response from both to ensure the data is valid before committing the enrollment to its own database. If a service is unreachable, it returns a 503 Service Unavailable.

8. API Gateway Explanation

An API Gateway acts as the single-entry point (Reverse Proxy) for all external clients.

- **Behaviour:** It intercepts requests at port 8080 and dynamically routes them to the correct internal container (ports 8000, 8001, or 8002) based on the URL path.

- **Benefit:** This hides the internal network topology from the client. The client doesn't need to know how many microservices exist or what ports they run on; it only talks to the Gateway.

9. API Versioning Strategy

We implemented **URI Routing Versioning**.

All endpoints include the version in the URL path (e.g., `/api/v1/students`). This strategy ensures backward compatibility. If a breaking change is introduced to the Student Service in the future (e.g., changing IDs from integers to UUIDs), we can deploy it under `/api/v2/students`. Older clients can safely continue using v1 until they are ready to migrate.

10. Docker/Containerization Steps

1. **Individual Dockerfiles:** A lightweight `python:3.11-slim` Dockerfile was created inside each of the four repository folders to define the exact environment, install `requirements.txt`, and expose port 8000 internally.
2. **Docker Compose:** A centralized `docker-compose.yml` file was created in the root workspace.
3. **Networking & Scaling:** The compose file maps the Gateway to host port 8080. Internal services omit static host ports (e.g., `ports: ["8000"]`) to allow Docker to dynamically assign ephemeral ports, enabling seamless horizontal scaling without port collisions.

11. Screenshots

```
PS P:\SS> docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
0af3bfc196d3	ss-student-service		"uvicorn main:app --..."	21 minutes ago	Up 5 seconds	0.0.0.0:62829->8000/tcp
cp, [::]:62829->8000/tcp	ss-student-service-2					
0a6a9e43f8f4	ss-student-service		"uvicorn main:app --..."	21 minutes ago	Up 4 seconds	0.0.0.0:62830->8000/tcp
cp, [::]:62830->8000/tcp	ss-student-service-3					
f7bad6f8bcf0	ss-api-gateway		"uvicorn main:app --..."	24 minutes ago	Up 3 seconds	0.0.0.0:8080->8000/tcp
p, [::]:8080->8000/tcp	ss-api-gateway-1					
60dbfd4b30df	ss-enrollment-service		"uvicorn main:app --..."	24 minutes ago	Up 4 seconds	0.0.0.0:8002->8000/tcp
p, [::]:8002->8000/tcp	ss-enrollment-service-1					
669287e3493f	ss-student-service		"uvicorn main:app --..."	24 minutes ago	Up 4 seconds	0.0.0.0:62831->8000/tcp
cp, [::]:62831->8000/tcp	ss-student-service-1					
e198227d571f	ss-course-service		"uvicorn main:app --..."	24 minutes ago	Up 5 seconds	0.0.0.0:8001->8000/tcp
p, [::]:8001->8000/tcp	ss-course-service-1					

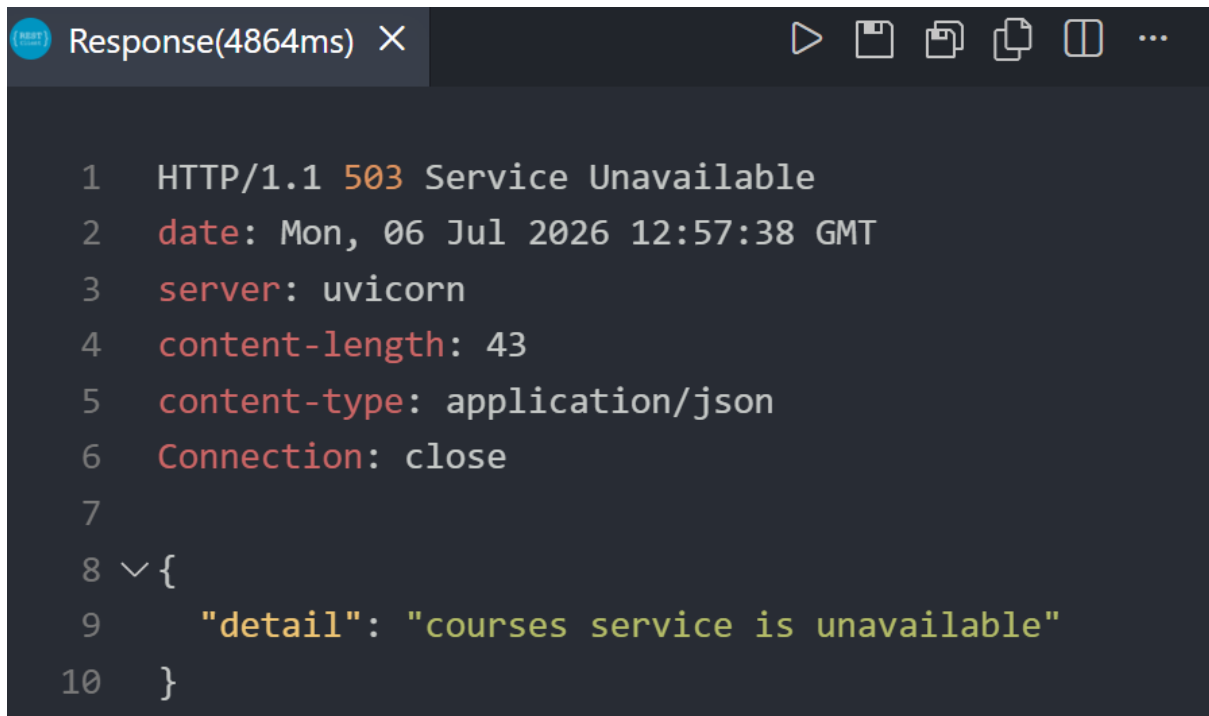
Fig: Docker containers after scaling student-service

```
Response(155ms) X
```

```

1  HTTP/1.1 201 Created
2  date: Mon, 06 Jul 2026 12:56:52 GMT, Mon, 06 Jul 2026 12:56:52 GMT
3  server: uvicorn
4  content-length: 65
5  content-type: application/json
6  Connection: close
7
8  {
9    "id": 3,
10   "name": "Rahul Sharma",
11   "email": "rahul.sharma@example.com"
12 }
```

Fig: HTTP response after a POST request from student-service



```
Response(4864ms) X

1 HTTP/1.1 503 Service Unavailable
2 date: Mon, 06 Jul 2026 12:57:38 GMT
3 server: uvicorn
4 content-length: 43
5 content-type: application/json
6 Connection: close
7
8 {
9   "detail": "courses service is unavailable"
10 }
```

Fig: HTTP response after sending a request to failing service

12. GitHub Repositories

The code has been separated into independent repositories to simulate a true microservices environment:

Main App: <https://github.com/PrabhanjanaSK/UniversityEnrollmentApp.git>

Submodule links:

- **Student Service:** <https://github.com/PrabhanjanaSK/student-service.git>
- **Course Service:** <https://github.com/PrabhanjanaSK/course-service.git>
- **Enrolment Service:** <https://github.com/PrabhanjanaSK/enrollment-service.git>
- **API Gateway:** <https://github.com/PrabhanjanaSK/api-gateway.git>